CUESTIONARIO RESUELTO — UNIDAD 6: SEGURIDAD EN SISTEMAS OPERATIVOS

> **Sistemas Operativos (UADE / Red Hat)**: resolución pedagógica de las 21 preguntas de `Consignas y cuestionario (Unidad 6).pdf`, basada en el apunte `Unidad 6.pdf` (Tanenbaum cap. 9).

Índice por módulos temáticos

Módulo	Tema	Preguntas
A	Entorno de seguridad: objetivos, amenazas y pérdida de datos	1 – 5
B	Criptografía: cifrado, claves, hash, firma digital y TPM	6 – 11
C	Mecanismos de protección (ACL)	12
D	Autenticación de usuarios	13 – 15
E	Defensas: defensa en profundidad, firewall, antivirus y firma de código	16 – 21

Mapa mental rápido

```
text
OBJETIVO                          AMENAZA
Confidencialidad de los datos ——  Exposición de los datos
Integridad de los datos ————————  Alteración de los datos
Disponibilidad del sistema ————   Negación del servicio (DoS)
Exclusión de usuarios externos —  Control del sistema por virus (zombies)

Autenticación = algo que SABÉS (contraseña) | algo que TENÉS (tarjeta) | algo que
SOS (biometría)
Defensa en profundidad = Firewall → Antivirus → Firma de código → Encarcelamiento →
IDS → Respaldos
```

MÓDULO A: Entorno de seguridad

Pregunta 1: Describa el concepto de confidencialidad de los datos desde la perspectiva de la seguridad.

> 💡 **En palabras simples (Para entenderlo de cero):** Es como tu **diario íntimo con candado**: solo lo leen las personas a las que vos se lo permitís. Un secreto tiene que **seguir siendo secreto**.

Explicación y Concepto de Cátedra:

- Es el **primer objetivo** de la seguridad: hacer que los **datos secretos permanezcan secretos**.
- Si el propietario decide que ciertos datos solo los pueden ver ciertas personas, el sistema debe **garantizar que las personas no autorizadas nunca tengan acceso**.
- Como mínimo, el **propietario** debe poder **especificar quién puede ver qué cosa**, y el sistema debe hacer cumplir esa especificación, idealmente **archivo por archivo**.
- **Amenaza asociada:** la **exposición de los datos**. Los atacantes que la buscan son los **intrusos pasivos**, que solo quieren *leer* lo que no les corresponde.
- Mecanismos que la protegen: permisos, ACLs, cifrado.

📌 **Ejemplo Práctico / En Linux:**

```
bash
chmod 600 sueldos.txt        # solo el dueño lee/escribe
ls -l sueldos.txt            # -rw------- 1 lucas lucas ...
sudo ls -l /etc/shadow       # ----------  root: los hashes de contraseñas son
confidenciales
gpg -c secreto.txt           # cifrado simétrico del archivo
```

Pregunta 2: Describa el concepto de integridad de los datos desde la perspectiva de la seguridad.

> 💡 **En palabras simples (Para entenderlo de cero):** Es como un **frasco con precinto de seguridad**: si alguien lo abrió o le cambió el contenido, no tiene que pasar desapercibido. Los datos tienen que **quedar como el dueño los dejó**.

Explicación y Concepto de Cátedra:

- Es el **segundo objetivo**: los **usuarios sin autorización no deben poder modificar datos** sin el permiso del propietario.
- "Modificar" incluye **cambiar** los datos, **eliminarlos** e **insertar datos falsos**.
- Si un sistema no garantiza que los datos queden **sin modificación hasta que el usuario decida**, **no tiene valor como sistema de información**.
- **Amenaza asociada:** la **alteración de los datos**, obra de los **intrusos activos**.
- Mecanismos: permisos de escritura, **funciones hash**, **firmas digitales** y comprobadores de integridad.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sha256sum contrato.pdf > contrato.sha256   # "huella" del archivo
sha256sum -c contrato.sha256               # contrato.pdf: OK  (o FAILED si fue
alterado)
rpm -V openssh-server                      # verifica que los archivos del paquete
no fueron modificados
```

Pregunta 3: Describa el concepto de disponibilidad del sistema desde la perspectiva de la seguridad.

> 💡 **En palabras simples (Para entenderlo de cero):** Un negocio tiene que **estar abierto cuando el cliente lo necesita**. Si alguien llena la puerta con 10.000 personas falsas, los clientes reales no pueden entrar. Eso es atacar la disponibilidad.

Explicación y Concepto de Cátedra:

- Es el **tercer objetivo**: **nadie debe poder perturbar el sistema para hacerlo inutilizable**.
- **Amenaza asociada:** la **negación del servicio (DoS, *Denial of Service*)**, un ataque muy común.
- **Ejemplo de cátedra:** si un servidor web tarda **100 µseg** en procesar cada petición, cualquiera que envíe **10.000 peticiones/segundo** lo satura, porque ocupa todo el tiempo de CPU solo en examinar y descartar peticiones.
- Hay modelos y tecnología razonables para defender la confidencialidad y la integridad, pero **es mucho más difícil frustrar los ataques de negación de servicio**.

📌 **Ejemplo Práctico / En Linux:**

```
bash
uptime                                  # carga del sistema
systemctl status httpd                  # ¿el servicio está disponible?
sudo firewall-cmd --add-rich-rule='rule service name=http limit value=25/m accept'
# limitar tasa
ulimit -u 200                           # limitar procesos por usuario (evita una
"fork bomb")
```

Pregunta 4: Describa el concepto de exclusión de los usuarios externos desde la perspectiva de la seguridad.

> 💡 **En palabras simples (Para entenderlo de cero):** Es evitar que un extraño **tome el control remoto de tu computadora** y la use como su "**soldado zombie**", por ejemplo para mandar spam, sin que vos te enteres.

Explicación y Concepto de Cátedra:

- Es el **último objetivo**: impedir que **usuarios externos tomen el control de las computadoras de otras personas** mediante **virus y otros medios**.
- Las máquinas comprometidas se convierten en **zombies**, dispuestas a obedecer las órdenes del usuario exterior. Un conjunto de zombies forma una *botnet*.
- **Uso típico:** enviar **spam** o lanzar ataques, de modo que **no se pueda rastrear** a la persona que está detrás.
- **Amenaza asociada:** el **control del sistema por parte de virus**.
- El apunte suma otro aspecto, la **privacidad**: proteger a las personas contra el mal uso de su información.

📌 **Ejemplo Práctico / En Linux:**

```
bash
ss -tunap                    # conexiones abiertas: ¿hay procesos raros "hablando"
afuera?
sudo lastb | head            # intentos de inicio de sesión fallidos
sudo firewall-cmd --list-all  # qué puertos están expuestos
```

Pregunta 5: ¿Cuáles son las causas más comunes de la pérdida accidental de datos?

> 💡 **En palabras simples (Para entenderlo de cero):** No todo es culpa de los hackers: se te puede **inundar la oficina**, **romper el disco** o podés **borrar algo sin querer**. La vacuna para los tres casos es la misma: **tener copias de respaldo**.

Explicación y Concepto de Cátedra: Además de los intrusos maliciosos, se pueden perder datos **por accidente**. Las causas comunes son:

1. **Accidentes y desastres naturales:** incendios, inundaciones, terremotos, guerras, disturbios o **ratas que roen cintas magnéticas**.
2. **Errores de hardware o software:** fallas de la CPU, discos o cintas ilegibles, errores de telecomunicaciones y errores en los programas.
3. **Errores humanos:** datos mal introducidos, una cinta o un CD-ROM mal montados, ejecutar el programa equivocado o perder un disco o una cinta.

- **Prevención: respaldos (backups) adecuados**, guardados **preferentemente lejos de los datos originales**.
- Dato clave de cátedra: **en la práctica, las pérdidas accidentales probablemente causan más daño que los intrusos**.

📌 **Ejemplo Práctico / En Linux:**

```
bash
tar -czf /backup/home_$(date +%F).tar.gz /home/lucas     # respaldo comprimido
rsync -av /datos/ usuario@servidor-remoto:/respaldo/     # copia lejos del original
sudo smartctl -H /dev/sda                                # salud del disco (falla de
HW)
```

MÓDULO B: Criptografía

Pregunta 6: ¿Cuál es el propósito principal del cifrado? Dé ejemplos de su aplicación en los sistemas operativos.

> 💡 **En palabras simples (Para entenderlo de cero):** Es como escribir una carta en un **idioma inventado** que solo vos y tu amigo saben traducir. Si alguien la intercepta, ve **garabatos sin sentido**.

Explicación y Concepto de Cátedra:

- **Propósito:** tomar un mensaje o archivo, el **texto simple (P)**, y convertirlo en **texto cifrado (C)**, de forma que **solo las personas autorizadas** sepan volver a convertirlo en texto simple. Para los demás es una **sucesión incomprensible de bits**.
- **Notación:** `C = E(P, K_E)` para cifrar y `P = D(C, K_D)` para descifrar. E y D son los algoritmos; K_E y K_D, las claves.
- **Principio de Kerckhoffs:** los **algoritmos deben ser públicos**; el **secreto reside exclusivamente en las claves**. Se opone a la "seguridad por oscuridad".

Ejemplos en sistemas operativos:

- **Cifrado de disco completo:** LUKS/dm-crypt en Linux, BitLocker en Windows, FileVault en macOS.
- **Conexiones remotas seguras:** SSH, TLS/HTTPS, VPN.
- **Archivos y correo:** GPG.
- **Protección de la swap** y de los directorios personales.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo cryptsetup luksFormat /dev/sdb1     # cifrado de partición con LUKS
lsblk -f | grep crypto_LUKS              # particiones cifradas
openssl enc -aes-256-cbc -pbkdf2 -in datos.txt -out datos.enc   # cifrar un archivo
```

Pregunta 7: Describa la criptografía de clave secreta. Dé ejemplos de su aplicación en los sistemas operativos.

> 💡 **En palabras simples (Para entenderlo de cero):** Es un **candado con una sola llave** que sirve para cerrar y para abrir. El problema es que **tenés que darle una copia de la llave** a la otra persona sin que nadie la robe en el camino.

Explicación y Concepto de Cátedra:

- También se la llama **simétrica**: usa la **misma clave** para cifrar y descifrar, o bien, dada la clave de cifrado, **es fácil obtener la de descifrado** (y viceversa).
- **Ejemplo de cátedra (sustitución monoalfabética):** con la clave `QWERTYUIOPASDFGHJKLZXCVBNM`, el texto `ATTACK` queda `QZZQEA`. La clave de descifrado es `KXVMCNOPHQRSZYIJADLEGWBUFT`.
    - Es **inútil en la práctica**: se rompe con **análisis estadístico** del idioma (la frecuencia de letras como e, t, o y de **bigramas** como th, in, er).
- Existen algoritmos simétricos **relativamente seguros si las claves son lo bastante largas**, como AES.
- ✅ **Ventaja:** es **eficiente**, porque requiere una cantidad razonable de cálculos.
- ❌ **Desventaja:** el **emisor y el receptor deben compartir la clave secreta** (el problema de la distribución de claves).

Ejemplos en sistemas operativos: LUKS/dm-crypt y BitLocker (AES), cifrado de archivos con `gpg -c`, cifrado del canal de una sesión SSH o TLS una vez negociada la clave, y cifrado de Wi-Fi (WPA2/AES).

📌 **Ejemplo Práctico / En Linux:**

```
bash
gpg --symmetric --cipher-algo AES256 informe.pdf   # cifra con contraseña (clave
simétrica)
gpg -d informe.pdf.gpg > informe.pdf               # descifra con la MISMA
contraseña
sudo cryptsetup luksDump /dev/sdb1 | grep -i cipher   # aes-xts-plain64
```

Pregunta 8: Describa la criptografía de clave pública. Dé ejemplos de su aplicación en los sistemas operativos.

> 💡 **En palabras simples (Para entenderlo de cero):** Es un **buzón de correo**: cualquiera puede **meter cartas por la ranura** (clave pública), pero **solo el dueño tiene la llave para abrirlo** (clave privada). Así no hace falta repartir ninguna llave secreta.

Explicación y Concepto de Cátedra:

- Resuelve el problema de la clave compartida usando **dos claves distintas**: una para **cifrar** (la **pública**) y otra para **descifrar** (la **privada**). Si se eligen bien, es **casi imposible deducir la privada a partir de la pública**.
- Se basa en la **asimetría computacional**: algunas operaciones son fáciles en un sentido y muy difíciles en el inverso. El apunte lo ilustra con elevar al cuadrado frente a calcular una raíz cuadrada.
- **RSA:** explota que es mucho más fácil **multiplicar números enormes** que **factorizarlos**, usando aritmética modular con números de cientos de dígitos.
- **Funcionamiento:** cada usuario genera un par (pública, privada) y **publica la pública**. Para enviar un secreto, se cifra con la **clave pública del receptor**, y **solo él** puede descifrarlo con su privada.
- ❌ **Desventaja:** es **mucho más lenta** que la simétrica. Por eso en la práctica se combinan: la clave pública sirve para intercambiar una clave simétrica de sesión.

Ejemplos en sistemas operativos: autenticación con claves SSH, certificados TLS/HTTPS (PKI), **firma de paquetes** (las claves GPG de RPM/DNF en Red Hat), Secure Boot y correo cifrado con GPG.

📌 **Ejemplo Práctico / En Linux:**

```
bash
ssh-keygen -t ed25519                # genera ~/.ssh/id_ed25519 (privada) y .pub
(pública)
ssh-copy-id usuario@servidor         # se entrega SOLO la pública
rpm -qa gpg-pubkey*                  # claves públicas de Red Hat instaladas en el
sistema
```

Pregunta 9: ¿Qué son las funciones de una vía y cómo se aplican en la seguridad de los sistemas operativos?

> 💡 **En palabras simples (Para entenderlo de cero):** Es como hacer un **licuado**: es facilísimo meter frutas y licuarlas, pero es **imposible "deslicuar"** el jugo para recuperar las frutas enteras.

Explicación y Concepto de Cátedra:

- Una **función de una vía** es **fácil de calcular en un sentido** (`y = f(x)`) pero **extremadamente difícil (inviable) de revertir**: teniendo solo `f(x)`, es imposible obtener `x`.
- En criptografía, manipula los bits de forma compleja (permutaciones, constantes y mezcla minuciosa). A eso se lo llama **función de hash criptográfica**.
- **Hash:** convierte cualquier dato en una **cadena de longitud fija** que funciona como su "**huella digital**". Sirve para garantizar **integridad y autenticidad**.
    - MD5 (16 bytes), SHA-1 (20 bytes), SHA-256 (32 bytes) y SHA-512 (64 bytes).

Aplicaciones en sistemas operativos:

1. **Almacenamiento de contraseñas:** UNIX/Linux **no guarda la contraseña**, sino su hash. Al iniciar sesión, calcula el hash de lo que escribió el usuario y lo compara. **Ni siquiera el superusuario puede ver las contraseñas.** Se le agrega un **salt** para frustrar los ataques de diccionario precalculado.
2. **Firmas digitales y firma de código**: se firma el hash, no el documento completo.
3. **Comprobadores de integridad** (antivirus, `rpm -V`, AIDE).
4. Verificar descargas (las sumas SHA-256 de una imagen ISO).

📌 **Ejemplo Práctico / En Linux:**

```
bash
echo -n "hola" | sha256sum      # siempre el mismo resultado; imposible volver a
"hola"
sudo grep lucas /etc/shadow     # lucas:$6$salt$hash...   ($6 = SHA-512 con salt)
```

Pregunta 10: ¿Qué es la firma digital? Dé ejemplos del uso de la firma digital en sistemas operativos.

> 💡 **En palabras simples (Para entenderlo de cero):** Es tu **firma a mano, pero imposible de falsificar**, que además funciona como **precinto**: demuestra **quién** firmó y que el documento **no se cambió** después. El firmante no puede decir después "yo no fui".

Explicación y Concepto de Cátedra:

- Permite firmar documentos digitales de modo que **el emisor no pueda repudiarlos** después (**no repudio**). El ejemplo del apunte es el cliente que ordena comprar acciones y luego niega haber enviado el correo.
- **Cómo se firma:**
    1. Se calcula el **hash** del documento (MD5, SHA-1, SHA-256...), que tiene longitud fija.
    2. El propietario aplica su **clave privada** al hash y obtiene `D(hash)`, el **bloque de firma**.
    3. Envía el documento junto con el bloque de firma.
- **Cómo se verifica:**
    1. El receptor calcula el hash del documento.
    2. Aplica la **clave pública del emisor** al bloque de firma: `E(D(hash)) = hash`.
    3. Si los dos hashes **coinciden**, el documento es auténtico e íntegro. Si **no coinciden**, se alteró el documento, la firma o ambos.
- **Ventaja:** la criptografía de clave pública, que es lenta, se aplica **solo al hash**, que es pequeño.
- **Distribución de claves públicas:** mediante **certificados** firmados por una **CA (Autoridad de Certificación)**, dentro de una **PKI (Infraestructura de Claves Públicas)**. Los navegadores ya traen precargadas las claves de las CA populares.

Ejemplos en sistemas operativos:

- **Paquetes RPM firmados** con GPG: `dnf` rechaza los paquetes con firma inválida.
- **Secure Boot:** se verifican las firmas del cargador de arranque y del kernel.
- **Módulos del kernel firmados** y **drivers firmados** en Windows.
- Actualizaciones del sistema operativo y certificados HTTPS.

📌 **Ejemplo Práctico / En Linux:**

```
bash
rpm -K paquete.rpm                          # digests signatures OK
gpg --detach-sign documento.pdf             # firmar
gpg --verify documento.pdf.sig documento.pdf   # Good signature from ...
grep gpgcheck /etc/yum.repos.d/*.repo       # gpgcheck=1
```

Pregunta 11: ¿Para qué se utiliza el chip llamado TPM? Dé ejemplos del uso del chip TPM en sistemas operativos.

> 💡 **En palabras simples (Para entenderlo de cero):** Es una **caja fuerte diminuta soldada a la placa madre**: guarda las llaves criptográficas para que **ningún programa pueda robarlas** y además sabe hacer operaciones de cifrado por su cuenta.

Explicación y Concepto de Cátedra:

- Toda la criptografía depende de las **claves**. Si se compromete la clave, **se compromete toda la seguridad**, así que hay que **almacenarlas de forma segura**.
- El **TPM (*Trusted Platform Module*, Módulo de Plataforma Confiable)** es un **criptoprocesador con almacenamiento no volátil** para guardar claves.
- **Funciones:** cifrar y descifrar bloques en memoria y **verificar firmas digitales**. Al hacerlo en hardware especializado es **más rápido**.
- **Controversia:** ¿quién controla el TPM y de quién protege? Puede usarse para que **solo se ejecute software autorizado** (por ejemplo, arrancar solo sistemas operativos firmados por el fabricante) o para impedir la piratería de música y películas (DRM).
- Según el apunte, el TPM **no ayuda a proteger contra ataques externos**: se enfoca en impedir que los usuarios hagan lo que no aprueba la entidad que lo controla.
- Hoy no siempre es un chip físico: existen **fTPM** (implementado en el firmware o en el procesador, como Intel PTT o AMD fTPM).

Ejemplos en sistemas operativos:

- **BitLocker** (Windows) guarda en el TPM la clave del disco. Microsoft también lo usó en **Palladium/NGSCB**.
- **Secure Boot / arranque medido (UEFI)**: el apunte lo presenta apoyado en el TPM; verifica las firmas de cada componente de arranque.
- **Windows 11** exige TPM 2.0.
- **Linux:** desbloqueo automático de LUKS con `systemd-cryptenroll` o Clevis.

📌 **Ejemplo Práctico / En Linux:**

```
bash
ls /dev/tpm*                                    # /dev/tpm0 /dev/tpmrm0
cat /sys/class/tpm/tpm0/tpm_version_major     # 2
sudo systemd-cryptenroll --tpm2-device=auto /dev/sda3   # atar la clave LUKS al TPM
mokutil --sb-state                              # SecureBoot enabled
```

MÓDULO C: Mecanismos de protección

Pregunta 12: ¿Para qué se usan las listas de control de acceso (ACL)?

> 💡 **En palabras simples (Para entenderlo de cero):** Es la **lista de invitados en la puerta de una fiesta**: cada salón (archivo) tiene su propia lista, que dice **quién puede entrar y qué puede hacer adentro**.

Explicación y Concepto de Cátedra:

- Sirven para **controlar qué sujetos (usuarios, grupos o dominios) pueden acceder a cada objeto (archivo, recurso) y con qué operaciones** (lectura, escritura, ejecución, etc.).
- **Origen:** la **matriz de protección** (dominios × objetos) es enorme y está casi vacía, así que no se guarda entera. Si se guarda **por columnas**, a **cada objeto se le asocia una lista ordenada** de dominios con sus permisos: eso es la **ACL**.
    - Ejemplo: `F1: A: RW; B: R` significa que A lee y escribe, B solo lee, y todo lo demás está prohibido.
- Los permisos se otorgan **por usuario, no por proceso**.
- Admiten **grupos** (entradas `UID, GID: permisos`), lo que introduce el concepto de **rol**. También admiten **comodines** (`*`) y **bloqueos selectivos** (por ejemplo, `virgil,*: (none); *,*: RW`, porque las entradas se evalúan en orden y gana la primera que coincide).
- Admiten permisos **genéricos** (destruir y copiar objeto) y **específicos** de cada objeto.
- ✅ **Ventaja frente a las capacidades:** permiten la **revocación selectiva** de permisos (basta editar la ACL) y no sufren problemas si se elimina el objeto.
- ⚠️ Si la ACL solo se revisa en el `open`, un archivo ya abierto conserva los derechos que tenía al abrirse.

📌 **Ejemplo Práctico / En Linux:**

```
bash
setfacl -m u:maria:rw informe.txt     # darle a maria lectura/escritura (más allá de
rwx clásico)
setfacl -m g:auditores:r informe.txt  # permiso para un grupo
getfacl informe.txt                   # ver la ACL completa
setfacl -x u:maria informe.txt        # revocación selectiva
```

MÓDULO D: Autenticación de usuarios

> 🔑 Los tres métodos de autenticación se basan en: **algo que el usuario sabe** (contraseña), **algo que tiene** (objeto físico) y **algo que es** (biometría).

Pregunta 13: Describa la autenticación mediante el uso de contraseñas.

> 💡 **En palabras simples (Para entenderlo de cero):** Es la **palabra secreta** del club: si la sabés, entrás. Se basa en **algo que sabés**.

Explicación y Concepto de Cátedra:

- Es la forma de autenticación **más usada**: el usuario escribe su **nombre de inicio de sesión y su contraseña**, y el sistema verifica que coincidan. La contraseña **no debe mostrarse** al escribirla.
- **Buena práctica de cátedra:** no avisar si el nombre de usuario es inválido antes de pedir la contraseña. Hay que decir solo que "**la combinación es incorrecta**", para no darle pistas al cracker.
- **Seguridad de contraseñas en UNIX:**
  - No se guardan en texto plano. Se aplica una **función de una vía (hash)** y se guarda el resultado. Al iniciar sesión se calcula el hash de lo ingresado y se compara. **Ni siquiera el superusuario puede ver las contraseñas.**
  - **Ataque de diccionario:** el cracker precalcula los hashes de palabras probables y los compara con el archivo de contraseñas.
  - **Defensa con salt:** a cada contraseña se le asocia un **número aleatorio de n bits** que se concatena antes de aplicar el hash. Así el cracker tiene que precalcular **2^n variantes** por cada palabra (UNIX clásico usaba n = 12).

    - Además, el archivo de hashes se hace **ilegible** (en Linux, `/etc/shadow`) y se agregan retrasos ante los intentos fallidos.

- **Contraseñas de un solo uso (OTP):** cambian en **cada inicio de sesión**. Si un intruso descubre una, **no le sirve** para la próxima vez.
- **Debilidad:** los usuarios eligen contraseñas débiles o las reutilizan.

📌 **Ejemplo Práctico / En Linux:**

```
bash
passwd                                  # cambiar la contraseña
sudo grep lucas /etc/shadow             # $6$<salt>$<hash>
sudo chage -l lucas                     # política de expiración
cat /etc/security/pwquality.conf        # complejidad mínima (minlen, dcredit...)
```

Pregunta 14: Describa la autenticación mediante el uso de un objeto físico.

> 💡 **En palabras simples (Para entenderlo de cero):** Es como la **llave de tu casa**: entrás porque **la tenés**. Hoy es una tarjeta, un token USB o el celular. Casi siempre se combina con un PIN, así que hace falta algo que tenés **y** algo que sabés.

Explicación y Concepto de Cátedra:

- Se comprueba **algo que el usuario tiene**, en lugar de algo que sabe (históricamente, las **llaves metálicas**). Hoy suele ser una **tarjeta de plástico** que se inserta en un lector.
- Normalmente se exige **además una contraseña o PIN**, para que no sirva una tarjeta perdida o robada. Es el caso del **cajero automático (ATM)**: tarjeta más PIN de 4 dígitos.
- **Tipos de tarjeta:**
    1. **De tira magnética:** guardan unos **140 bytes**. Son **riesgosas**, porque el equipo para leerlas y escribirlas es barato y fácil de conseguir.
    2. **De chip**, que pueden ser:
        - **De valor almacenado:** tienen una pequeña memoria pero **no tienen CPU**, así que el valor lo modifica la CPU del lector.
        - **Inteligentes (*smart cards*):** son una **pequeña computadora a prueba de falsificaciones**, con CPU y a veces coprocesador criptográfico. Autentican al usuario mediante un **protocolo de desafío-respuesta** con la computadora central y **no necesitan conexión en línea** con el banco.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo dnf install opensc pcsc-lite     # soporte de smart cards en RHEL
pkcs11-tool --list-slots              # lectores/tarjetas detectadas
# Tokens FIDO2/YubiKey como 2.º factor para SSH:
ssh-keygen -t ed25519-sk
```

Pregunta 15: Describa la autenticación biométrica.

> 💡 **En palabras simples (Para entenderlo de cero): Tu propio cuerpo es la llave**: tu huella, tu iris, tu cara o tu voz. Se basa en **algo que sos** y no se puede olvidar ni prestar.

Explicación y Concepto de Cátedra:

- Mide **características físicas del usuario difíciles de falsificar**, como huellas digitales, voz, iris, longitud de los dedos o la dinámica de la firma.
- **Dos fases:**
    1. **Inscripción:** se miden y digitalizan las características y se guarda un **registro** asociado al usuario, en una base central o en una tarjeta inteligente.
    2. **Identificación:** el usuario da su **nombre de inicio de sesión**, se vuelve a medir y, si coincide con lo registrado, se acepta.
- Se pide el nombre porque **las mediciones nunca son exactas** (es difícil indexarlas) y porque **dos personas podrían parecerse**. Es más firme comparar contra un usuario específico.
- **Requisitos de la característica:** suficiente **variabilidad** entre personas (el color de pelo no sirve) y que **no cambie con el tiempo** (la voz cambia con un resfrío, la cara con la barba).
- **Compromiso de diseño:** hay que fijar un umbral y decidir qué es peor, **rechazar a un usuario legítimo** (falso rechazo) o **aceptar a un impostor** (falsa aceptación).
- **Ejemplos y ataques:**
    - **Longitud de los dedos:** se puede engañar con moldes de yeso.
    - **Iris:** es único incluso entre gemelos, se extraen sus rasgos con **ondículas de Gabor** y se comprimen a 256 bytes. Puede ser suplantado con fotos, así que se usan contramedidas como un flash para ver si la pupila se contrae. Se usa en el aeropuerto de Ámsterdam desde 2001.
    - **Análisis de firmas:** conviene comparar **los movimientos, la velocidad y la presión** de la pluma, no solo la forma.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo dnf install fprintd fprintd-pam
fprintd-enroll                          # fase de INSCRIPCIÓN de la huella
fprintd-verify                          # fase de IDENTIFICACIÓN
sudo authselect enable-feature with-fingerprint   # integrarlo con PAM en RHEL
```

MÓDULO E: Defensas

Pregunta 16: Describa el concepto de defensa en profundidad.

> 💡 **En palabras simples (Para entenderlo de cero):** Una casa bien protegida tiene **reja con púas, sensores de movimiento en el jardín, dos candados en la puerta y alarma adentro**. El ladrón tendría que vencer **todas** las barreras, no solo una.

Explicación y Concepto de Cátedra:

- Es **uno de los conceptos más importantes** del diseño de sistemas seguros: tener **varios niveles de seguridad**, de modo que **si se viola uno, todavía queden otros**.
- Cada técnica vale por sí sola, pero para comprometer el sistema el atacante **tendría que vencerlas todas**.
- Según el apunte, las defensas **no son estrictamente jerárquicas**, aunque se pueden ordenar de las más generales y externas a las más específicas:
    1. **Firewall**, para que los intrusos no entren.
    2. **Antivirus**, si el firewall falla.
    3. **Firma de código**, para ejecutar solo software confiable.
    4. **Encarcelamiento**, para vigilar las llamadas al sistema.
    5. **Detección de intrusos (IDS)** basada en modelos.
    6. A eso se suman la autenticación, las ACLs, el cifrado y los **respaldos**.

📌 **Ejemplo Práctico / En Linux (capas en Red Hat):**

```
bash
sudo firewall-cmd --state          # capa 1: firewall (firewalld/nftables)
getenforce                         # capa 2: SELinux en modo Enforcing (control de
acceso obligatorio)
grep gpgcheck /etc/dnf/dnf.conf    # capa 3: firma de paquetes
sudo ausearch -m avc               # capa 4: auditoría
ls -l /etc/shadow                  # capa 5: permisos y contraseñas con hash
```

Pregunta 17: Describa el propósito y funcionamiento de un firewall.

> 💡 **En palabras simples (Para entenderlo de cero):** Es el **puente levadizo de un castillo con foso**: todos los que entran o salen **tienen que pasar por ahí**, y un guardia revisa a cada uno según una lista de reglas.

Explicación y Concepto de Cátedra:

- **Propósito:** una computadora conectada a Internet enfrenta **peligros entrantes** (crackers, virus, spyware y otro malware) y **salientes** (fuga de información confidencial como tarjetas, contraseñas o datos corporativos). El firewall busca **mantener los bits "buenos" adentro y los "malos" afuera**.
- **Funcionamiento: todo el tráfico entrante y saliente pasa obligatoriamente por el firewall**, que lo filtra según **reglas** configuradas por el administrador sobre qué puede entrar y qué puede salir.
- **Variedades:**
    - **De hardware:** un equipo dedicado entre el proveedor y la LAN. Es el que suelen usar las empresas.
    - **De software (firewall personal):** son **filtros conectados al código de red dentro del kernel del sistema operativo**. Es el que suelen usar los individuos.
- **Tipos según inspección:**
    - **Sin estado (*stateless*):** decide paquete por paquete mirando **solo el encabezado**: IP de origen y destino, **puertos** de origen y destino, tipo de servicio y protocolo. Ejemplo: permitir el puerto 80 hacia 207.68.160.190, el 25 hacia .191 y el 21 hacia .192, y **descartar en silencio** todo lo demás.
    - **Con estado (*stateful*): registra las conexiones y su estado**. Es mejor contra los ataques relacionados con el establecimiento de conexiones.
    - **Con IDS:** inspecciona también el **contenido** de los paquetes en busca de material sospechoso.

- **Limitaciones:** no protege si se explota un servicio permitido (por ejemplo, un desbordamiento de búfer en Apache por el puerto 80) ni si el usuario abre puertos para un juego con un troyano. **Cada puerto abierto es un agujero más.**

📌 **Ejemplo Práctico / En Linux (Red Hat):**

```
bash
sudo firewall-cmd --list-all                          # reglas de la zona activa
sudo firewall-cmd --permanent --add-service=http      # permitir puerto 80
sudo firewall-cmd --permanent --remove-service=ftp    # cerrar puerto 21
sudo firewall-cmd --reload
sudo nft list ruleset                                 # reglas reales en el kernel
(netfilter)
```

Pregunta 18: ¿Es correcto afirmar que un firewall forma parte del sistema operativo? Justifique su respuesta.

> 💡 **En palabras simples (Para entenderlo de cero): Depende del tipo**. Un firewall de **hardware** es un aparato aparte, como un guardia en la entrada del barrio. Uno de **software** vive **dentro del sistema operativo**, como un guardia en la puerta de tu casa.

Explicación y Concepto de Cátedra:

FALSO como afirmación general (es *parcialmente* cierto: depende del tipo de firewall).

Justificación:

1. **Firewall de hardware:** es un **dispositivo independiente**, ubicado entre la conexión del proveedor y la LAN, con su propio firmware y su propia interfaz web de configuración. **No forma parte del sistema operativo** de las computadoras que protege.
2. **Firewall de software (personal):** según el apunte, son **filtros que se conectan al código de red dentro del kernel del sistema operativo**. En ese caso el **mecanismo de filtrado sí está integrado al sistema operativo**.
    - En Linux, el filtrado lo hace **netfilter**, que es parte del kernel. `nftables` / `iptables` y el demonio `firewalld` son las herramientas de espacio de usuario para configurarlo.
    - Windows incluye **Windows Defender Firewall**.
3. Aun así, conceptualmente el firewall es un **mecanismo de defensa** (una capa de la defensa en profundidad) y **no una función esencial del sistema operativo** como la gestión de procesos o de memoria. El sistema puede funcionar sin él, y además existen firewalls de terceros.

📌 **Ejemplo Práctico / En Linux:**

```
bash
lsmod | grep -E "nf_tables|nf_conntrack"   # módulos de netfilter: el filtrado vive
en el KERNEL
systemctl status firewalld                 # herramienta de espacio de usuario que
lo configura
```

Pregunta 19: Describa brevemente el propósito y el funcionamiento de un software antivirus.

> 💡 **En palabras simples (Para entenderlo de cero):** Es el **sistema inmune** de la computadora: conoce las "caras" de los virus (sus firmas), revisa que nada haya cambiado sin permiso y vigila a los programas que se portan raro.

Explicación y Concepto de Cátedra:

- **Propósito:** es la **siguiente línea de defensa cuando falla el firewall**. Detecta y elimina **malware**: virus, y muchos también gusanos y spyware. En sentido estricto se los llama programas **antimalware**.
- **Técnicas de funcionamiento:**
    1. **Exploradores de virus (por firmas):** los laboratorios infectan un **archivo señuelo (*goat file*)**, obtienen el código del virus y lo agregan a una **base de datos de virus conocidos**. El antivirus explora los ejecutables, el **MBR**, los sectores de arranque, la memoria flash y la CMOS buscando esas **firmas**.
        - Usa **búsqueda difusa** para detectar variantes, con el riesgo de **falsos positivos**.
        - También controla fechas y **longitudes de archivos**.
        - El virus contraataca con compresión, cifrado propio con una clave distinta en cada copia, o con **virus polimórficos**. El antivirus responde buscando la **rutina de descifrado**, que siempre está en texto plano. Es el "juego del gato y el ratón".
    2. **Comprobadores de integridad:** con el disco limpio, calculan una **suma de comprobación o hash** de cada ejecutable y la guardan. Después vuelven a calcularla y comparan. Conviene **firmar digitalmente** ese archivo de sumas para que el virus no lo altere.
    3. **Comprobadores del comportamiento:** el antivirus **reside en memoria** y **atrapa las llamadas al sistema**, alertando sobre acciones sospechosas, como sobrescribir el sector de arranque, la memoria flash o un ejecutable.

- Hay que **actualizarlo** periódicamente y ejecutar exploraciones frecuentes.

📌 **Ejemplo Práctico / En Linux:**

```
bash
sudo dnf install clamav clamav-update
sudo freshclam                 # actualiza la base de firmas
clamscan -r /home/lucas        # explorador por firmas
sudo aide --check              # comprobador de integridad (AIDE, incluido en RHEL)
```

Pregunta 20: ¿Es correcto afirmar que un antivirus forma parte del sistema operativo? Justifique su respuesta.

> 💡 **En palabras simples (Para entenderlo de cero):** No. El antivirus es como un **guardia de seguridad contratado aparte**: protege el edificio, pero no es parte de la estructura del edificio.

Explicación y Concepto de Cátedra:

FALSO (en sentido estricto).

Justificación:

1. El antivirus es una **aplicación de seguridad**, normalmente desarrollada y vendida por **terceros** (el apunte menciona que hay un **mercado** de software antivirus y empresas que compiten por el tamaño de sus bases de datos). **No es una función esencial del sistema operativo**: el sistema administra procesos, memoria, archivos y E/S, y funciona perfectamente sin antivirus.
2. Es una **capa más de la defensa en profundidad**, que actúa sobre el sistema operativo. **Usa sus servicios**, como las llamadas al sistema para leer archivos o interceptar syscalls. No forma parte de su núcleo (no pertenece a la TCB).
3. El propio apunte señala que un antivirus podría tener que **pasar por alto al sistema operativo** (leer el disco directamente) para no ser engañado por un virus, lo que muestra que es un programa distinto del sistema.

Matiz: algunos sistemas operativos **incluyen o preinstalan** un antivirus (por ejemplo, **Microsoft Defender** en Windows) y algunos componentes funcionan como **drivers en modo kernel** para interceptar operaciones. Aun así, son **componentes agregados de seguridad**, no parte del núcleo del sistema operativo. En Linux/Red Hat no viene por defecto: se instala aparte (por ejemplo, ClamAV).

📌 **Ejemplo Práctico / En Linux:**

```
bash
rpm -q clamav || echo "No instalado: el SO funciona igual sin antivirus"
```

Pregunta 21: ¿Qué es la firma de código y qué ventajas presenta?

> 💡 **En palabras simples (Para entenderlo de cero):** Es el **sello de fábrica inviolable** de un programa: te garantiza **quién lo hizo** y que **nadie lo tocó** desde que salió de la fábrica. Si el sello está roto, no lo usás.

Explicación y Concepto de Cátedra:

- Es un método para **impedir que el malware entre**: **ejecutar solo software de distribuidores confiables y sin modificaciones**. Responde dos preguntas: ¿el software **proviene realmente** del distribuidor correcto? y ¿**se modificó** desde que salió de fábrica?
- Se basa en la **criptografía de clave pública** y en la **firma digital**:
    1. **Firma:** el distribuidor genera un par de claves (publica la pública y guarda la privada), calcula el **hash del código** (MD5, SHA-1 o SHA-256, de 128, 160 o 256 bits) y lo **cifra con su clave privada**. Esa firma **acompaña al software**.
    2. **Verificación:** el usuario calcula el hash del software recibido, **descifra la firma con la clave pública** del distribuidor y compara. Si coinciden, el código es **genuino**; si no, se **rechaza como falsificación**.
- La clave pública suele venir en un **certificado firmado por una CA**. Si la CA es desconocida, el navegador o el sistema le preguntan al usuario si lo acepta.

Ventajas:

1. **Autenticidad:** garantiza el **origen** del software.
2. **Integridad:** detecta **cualquier modificación**, por ejemplo un troyano insertado en el camino.
3. **No repudio:** el distribuidor no puede negar que lo firmó.
4. Es **muy difícil de falsificar**: no se puede alterar el código manteniendo el mismo hash ni generar una firma válida sin la clave privada.
5. **Se verifica automáticamente** (navegadores, gestores de paquetes, Secure Boot) y **reduce mucho la probabilidad de problemas** al ejecutar solo programas, complementos, drivers y controles ActiveX firmados.

- **Límite:** la firma no garantiza que el código sea correcto o benigno, solo de quién viene. Por eso el apunte sugiere complementarla con el **encarcelamiento**, según el principio "confiar, pero verificar".

📌 **Ejemplo Práctico / En Linux (Red Hat):**

```
bash
rpm -K htop-*.rpm                          # verifica la firma GPG del paquete
rpm -qi bash | grep Signature              # firma del paquete instalado
sudo dnf install paquete                   # dnf rechaza paquetes con firma inválida
(gpgcheck=1)
modinfo -F signer xfs                      # módulo del kernel firmado
```

✅ Checklist de calidad

- ✓
  Se respondieron las **21 preguntas**, agrupadas por módulos y con la numeración original.
- ✓
  Cada respuesta tiene **analogía**, **explicación de cátedra** y **ejemplo en Linux**.
- ✓
  Las preguntas de afirmación (18 y 20) empiezan con **FALSO** y su justificación.
- ✓
  Términos clave preservados: confidencialidad/integridad/disponibilidad, DoS, zombies, Kerckhoffs, simétrica/asimétrica, RSA, hash, salt, bloque de firma, CA/PKI, TPM, ACL, smart card, inscripción/identificación, defensa en profundidad, firewall con y sin estado, firmas de virus, comprobadores de integridad y comportamiento, firma de código.

📚 Bibliografía

- Tanenbaum, A.: *Sistemas Operativos Modernos*, cap. 9 (Seguridad): 9.1, 9.2, 9.3, 9.4 y defensas.
- Apunte de cátedra: `Unidad 6.pdf`.